TECHNICAL SPECIFICATION

Loan & Inventory Management System

Technical Specification Audit & Implementation Blueprint

PREPARED BY

Godfrey Shora

Lead Developer

David Erick Mwijage

Architecture & Systems Engineering

Field	Detail
Product	Phone Loan & Inventory Management System
Platform	Flutter (Dart) + Supabase (PostgreSQL)
Document ID	LIMS-SDD-2026-001
Version	1.0 · 29 September 2026
Status	Approved for Execution
Classification	Internal — Engineering
Stack Audit	37 Dart files · ~6,430 LOC · 12 DB tables
Lead Developer	Godfrey Shora
Architecture & Systems	David Erick Mwijage

Confidential — prepared for the engineering team. Not for external distribution.

Contents

1. Executive Summary 4

2. Document Purpose, Scope & Audience 5

2.1 In Scope 5

2.2 Out of Scope 5

2.3 Audience 5

3. System Overview — Current Architecture 6

3.1 Repository Layout 6

3.2 Front-End Stack 6

3.3 Back-End & Database 6

4. Specification Compliance Audit — 20 Requirements 8

4.1 Audit Summary 9

5. Critical Gap Analysis 10

5.1 Gap 1 — No Live Data Layer 10

5.2 Gap 2 — No Immutable Inventory Ledger 10

5.3 Gap 3 — No Offline Persistence or Sync Queue 10

5.4 Gap 4 — Thirteen Dead Navigation Targets 10

5.5 Gap 5 — Transaction Mathematics Absent 10

5.6 Gap 6 — AI Layer Has No Data Access 11

6. Target Database Design — Schema Extensions 12

6.1 inventory_movements — Mandatory 12

6.2 sync_operations — Mandatory 12

6.3 loan_schedules — Recommended 13

6.4 alert_rules — Recommended 13

7. Offline-First Sync Architecture 14

7.1 Local Store 14

7.2 Write Path — Local First, Always 14

7.3 Sync Worker . . . 14
7.4 Conflict Resolution . . . 14
7.5 Sync Surface in the UI . . . 15

8. Security & Access Control . . . 16

8.1 Authentication . . . 16
8.2 Row Level Security . . . 16
8.3 Audit Integrity . . . 16
8.4 Attendance Anti-Tampering . . . 16

9. Implementation Roadmap . . . 17

9.1 Phase 1 — Database Foundation & Supabase Integration . . . 17
9.2 Phase 2 — Transaction Flows . . . 17
9.3 Phase 3 — Stocktake, Daily Closing & Reporting . . . 18
9.4 Phase 4 — AI Intelligence Layer . . . 18

10. Delivery Ownership Matrix . . . 20

11. Acceptance Criteria & Test Coverage . . . 21

12. Risk Register & Mitigations . . . 22

13. Muhtasari wa Kiswahili . . . 23

13.1 Hali ya Sasa ya App . . . 23
13.2 Mpango wa Utekelezaji . . . 23
13.3 Kanuni ya Msingi . . . 23

Appendix A — Source File Inventory . . . 24

Appendix B — Dependency Additions . . . 26

Appendix C — Glossary . . . 27

Sign-Off & Approval . . . 28

1. Executive Summary

The Loan & Inventory Management System (LIMS) is a multi-branch mobile phone retail platform built to run the full commercial cycle of a phone dealership: box-level intake, per-IMEI inventory tracking, retail sales, installment loans, inter-branch transfers, staff attendance, daily reconciliation, management reporting, and an AI intelligence layer for branch managers.

The application has been constructed **UI-first**. The Flutter front end is genuinely impressive in scope: 37 Dart source files, approximately 6,430 lines of hand-written application code, a complete go_router navigation graph, a localisation layer, and 16 reusable design-system widgets. Every screen a branch operator needs — scan, sell/loan, receive, transfer, attendance, daily close — already exists and renders.

What does not exist yet is the **engine underneath it**. This audit was conducted by direct inspection of the codebase at the path recorded in the Document Control table. The findings are unambiguous:

- `api_service.dart` is a pure mock. Every method returns a hard-coded stub and the live Supabase calls sit beneath them, commented out.
- `backend/lib/supabase_client.dart` contains the correct query shapes but its credentials are still the literal placeholders `YOUR_SUPABASE_URL` and `YOUR_SUPABASE_ANON_KEY`.
- The database schema defines 12 tables with sound foreign keys and indexes, but is missing the two tables that make the system auditable: an immutable inventory ledger and an offline sync queue.
- Thirteen navigation targets in the Admin and Reports dashboards are wired to `onTap: () {}` — dead buttons visible to users.
- There is no local persistence package of any kind in `pubspec.yaml`. The README promises an offline-first architecture that the front end has not yet begun to implement.
- There is no barcode scanning package, no PDF/CSV export package, and no AI SDK. Consequently the scanner, the reporting engine, and the Gemini assistant are all shape without substance.

This document converts those findings into an executable engineering plan. It records the audit verdict against all 20 specification requirements, specifies the exact schema additions required, defines the offline sync contract, and sequences delivery into four phases with a named owner for every workstream.

> **AUDIT VERDICT**
> The LIMS is a complete, well-architected presentation layer with an unfinished data layer. It is not a rewrite — it is a wiring, persistence and intelligence build. Estimated 4 phases to commercial readiness, with Phase 1 (database foundation and Supabase integration) unlocking the other three.

2. Document Purpose, Scope & Audience

This is the controlling technical document for LIMS development. It serves three functions: an audit record of what exists today, a specification of what must be built, and a delivery contract between the two engineers executing the build — **Godfrey Shora** (Lead Developer) and **David Erick Mwijage** (Architecture & Systems).

2.1 In Scope

- Full 20-point specification compliance audit against the live codebase.
- Database design extensions: `inventory_movements`, `sync_operations`, `loan_schedules`, `alert_rules`.
- Supabase integration and authentication wiring.
- Offline-first persistence and sync-worker design.
- Transaction mathematics for sales, loans and amortisation.
- Scanning, reporting and export capability.
- Gemini AI integration with function calling against live store data.
- Security architecture: Row Level Security policies and session handling.
- Phase sequencing, ownership allocation, acceptance criteria and risk register.

2.2 Out of Scope

- Hardware procurement and USB scanner device selection.
- Commercial pricing, licensing and customer onboarding.
- Production hosting cost modelling beyond Supabase tier selection.
- Play Store and App Store storefront submission workflows.

2.3 Audience

Primary readers are the two development owners. Secondary readers are branch operations leadership who need to understand what capability becomes available in each phase, and any future engineer inheriting the codebase.

3. System Overview — Current Architecture

The codebase is organised as two sibling projects under a single repository root, with a product specification document alongside them.

3.1 Repository Layout

```
LOAN APP/
frontend/ Flutter application (37 Dart files, ~6,430 LOC)
lib/main.dart App entry, Riverpod scope
lib/core/routing/ app_router.dart — go_router graph, 12 routes
lib/core/services/ api_service.dart — MOCK DATA PROVIDER
lib/core/theme/ app_theme.dart, app_localizations.dart
lib/core/widgets/ 16 reusable design-system widgets
lib/core/constants/ app_constants.dart
lib/features/<feature>/presentation/ 15 feature screens
pubspec.yaml 4 runtime dependencies only
test/widget_test.dart Default scaffold test
backend/
lib/supabase_client.dart Query layer (credentials are placeholders)
supabase/schema.sql 12 tables, 8 indexes, FK constraints
.env.example
README.md, specification document
```

3.2 Front-End Stack

Layer	Implementation	Assessment
State management	flutter_riverpod ^2.4.9	Correct choice. Providers present but no repository layer between provider and mock.
Navigation	go_router ^13.0.1	Complete. 12 routes of which 8 sit outside the shell — no persistent branch context.
Localisation	flutter_localizations + intl ^0.20.2	Present. Swahili and English strings exist.
Data transport	supabase_flutter ^2.3.0	Declared in pubspec but never invoked from the front end.
Design system	16 custom widgets	Strong. metric_card, status_badge, discrepancy_card, sync_status already exist.
Persistence	None	CRITICAL GAP. No sqflite, drift or hive. Offline-first is unimplemented.
Scanning	None	CRITICAL GAP. No mobile_scanner or barcode package. Scan screens are keyboard-only.
Export	None	CRITICAL GAP. No pdf, printing or csv package.
AI	None	CRITICAL GAP. AIScreen does keyword matching on local strings only.
Testing	widget_test.dart scaffold	Only the default counter test exists.

3.3 Back-End & Database

The supplied schema is well-formed. It uses UUID primary keys via `uuid-ossp`, `TIMESTAMP WITH TIME ZONE` for all temporal columns, explicit foreign key relationships, and eight targeted indexes including `idx_phones_imei`, `idx_phones_status`, `idx_phones_branch` and `idx_sales_date` — precisely the access paths a phone dealership hammers.

The existing twelve tables are:

```
branches employees phones customers
sales loans loan_payments stock_transfers
transfer_items attendance daily_closing notifications
audit_logs
```

The gap is architectural rather than structural. `phones.status` is a mutable single-value column, which means the system knows what a phone **is** but not what happened to it. There is no append-only event record, so a dispute over a device's location or a loan's history cannot be reconstructed. And `stock_transfers` and `transfer_items` are written directly, with no outbox to survive a dead network.

4. Specification Compliance Audit — 20 Requirements

Each requirement is assessed against the live codebase. Status values are **Complete**, **Partial** (UI exists, logic absent), and **Missing** (no implementation found).

No	Requirement	Status	Current State in Codebase	Gap — What Must Be Built
01	Product Overview & Architecture	Partial	UI correctly split into Branch UI and Admin UI. Router graph complete.	Supabase backend not connected to front end. `api_service.dart` returns mock data only.
02	Problem to Solve	Partial	IMEI, Transfers, Reports and AI screens all render.	No local database or outbox. No real IMEI uniqueness validation while offline.
03	User Experience	Partial	All branch screens present (Scan, Sell/Loan, Receive, Transfer, Attendance, Daily Close) plus Admin dashboard.	13 admin/report menu targets are dead: `onTap: () {}` on Audit Logs, Settings, Employees and report actions.
04	Box / Batch Receiving	Partial	`receive_stock_screen.dart` models box BX-1024, brand counts, discrepancy card.	Static sample data. Needs real batch intake of an IMEI manifest with expected-vs-received reconciliation.
05	Inventory & Transfer Engine	Partial	`stock_transfers` and `transfer_items` exist in schema.sql.	Missing `inventory_movements` — an immutable ledger so every device state change is a traceable event.
06	Sales & Loan Flow	Partial	`sell_loan_screen.dart` has sale, loan, deposit and repayment forms.	No formal installment amortisation schedule and no database transaction binding.
07	Stocktake & Daily Closing	Partial	`daily_closing_screen.dart` computes Opening, Received, Transfers, Sold, Expected vs Physical.	Not linked to a real scan that diffs shelf stock against the database to produce a discrepancy report.
08	Attendance	Partial	`attendance_screen.dart` has a check-in action.	Requires authoritative server timestamp and employee session recording to prevent device-clock tampering.
09	Notifications & Alerts	Partial	`notifications_screen.dart` renders a list.	Alerts are a static array. Must be generated by database rules (low stock, missing report, discrepancy, overdue loan).
10	Reporting Engine	Partial	`reports_screen.dart` and `branch_performance_screen.dart` with basic charts.	Needs PDF/Excel/CSV export and live queries against database tables instead of literals.
11	AI Intelligence Layer	Partial	`ai_screen.dart` has chat with keyword detection for sales, stock, loan, IMEI.	Must connect to Google Gemini with function calling so it answers from live store data and can never write directly.
12	Alert & Automation Engine	Missing	No rules engine of any kind.	Build `alert_rules` table plus a scheduled evaluator that raises notifications on threshold breach.
13	Offline-First Architecture	Missing	Claimed in README. No persistence package in pubspec.	Implement SQLite/Drift/Hive outbox queue keyed by `client_operation_id` plus a background sync worker.
14	Security Architecture	Partial	`login_screen.dart` exists; `signIn` wrapper written.	Needs Supabase Auth sessions, token expiry handling and Row Level Security policies on every table.
15	Database Design	Partial	`schema.sql`: 12 tables, good indexes and foreign keys.	Add `inventory_movements` and `sync_operations`. Recommend `loan_schedules` and `alert_rules`.

No	Requirement	Status	Current State in Codebase	Gap — What Must Be Built
16	API & Real-time Layer	Partial	Supabase client setup present in backend.	Front end has zero `supabase_flutter` calls — all commented out. Needs repository layer and realtime subscriptions.
17	Performance	Complete	App is light and cold-starts under 2 seconds.	Add camera barcode scanning and USB wedge-mode scanner support to the scan flow.
18	Testing & Acceptance	Missing	Only the default `widget_test.dart`.	Needs integration tests covering end-to-end sale, transfer and stocktake flows.
19	Phases Delivery	Complete	Phases defined in specification.	Now sequenced in Section 9 of this document with owners and acceptance gates.
20	Final System Model	Complete	Model matches the original design description.	Approved. Ready for implementation.

4.1 Audit Summary

Status	Count	Requirements
Complete	3	17 Performance · 19 Phases Delivery · 20 Final System Model
Partial	14	01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 14, 15, 16
Missing	3	12 Alert & Automation Engine · 13 Offline-First Architecture · 18 Testing & Acceptance

Three of twenty requirements are satisfied outright. Fourteen are partial — and in every partial case the missing half is the data layer, not the interface. Three are entirely absent: the rules engine, offline persistence, and integration testing.

> **READ THIS FIRST**
> The fourteen partial failures share one root cause: **no repository layer**. Every screen calls `ApiService`, which returns stubs. Build the repository layer and Supabase wiring once, and those fourteen requirements move together.

5. Critical Gap Analysis

The audit surfaces six gaps that block commercial operation. They are ordered by blast radius — Gap 1 unblocks the most downstream work.

5.1 Gap 1 — No Live Data Layer

`api_service.dart` is a singleton whose every method returns fabrication. `login()` waits one second then returns a user named John at Kariakoo Branch. `getPhones()` returns an empty list. `createSale()` echoes its input back with `id: '1'`. Approximately 30 lines of commented Supabase calls sit beneath them, ready but unwired.

The correct fix is not to uncomment those lines inside the service. Introduce a `SupabaseRepository` implementing an abstract `LimsRepository`, and have `apiServiceProvider` select between the live implementation and a seeded `MockRepository` by environment flag. This preserves demonstrability while making the live path the production default.

5.2 Gap 2 — No Immutable Inventory Ledger

A dealership's most expensive failure mode is an untraceable device. The current schema records only current state. If a phone's branch changes, the prior location is lost. If a loan is disputed, the payment sequence must be inferred from timestamps.

`inventory_movements` closes this. It is append-only by policy: inserts only, no updates, no deletes, enforced by RLS. Every intake, sale, loan issuance, transfer dispatch, transfer receipt, return and write-off writes one row. Device location and history then become derivable projections rather than mutable facts.

5.3 Gap 3 — No Offline Persistence or Sync Queue

Branch connectivity in Tanzanian retail environments is intermittent. A sales terminal that cannot complete a transaction when the link drops is unusable. The README promises offline-first; the implementation has no persistence package at all.

The remedy is a local SQLite database (via `drift` for typed queries) holding a mirrored subset of the catalogue plus a durable outbox. Every mutation is written locally first with a client-generated `client_operation_id` UUID, then enqueued. A sync worker drains the queue when connectivity returns. Server-side idempotency on that UUID guarantees replay safety.

5.4 Gap 4 — Thirteen Dead Navigation Targets

Direct inspection found 13 occurrences of `onTap: () {}` across two files:

File	Occurrences	Affected Actions
`features/admin/presentation/admin_dashboard_screen.dart`	9	Audit Logs, Employees, Settings, Branch Management, and five secondary admin tiles
`features/reports/presentation/reports_screen.dart`	4	Export and report-generation actions

These are visible to end users and read as defects. Each must either be implemented or removed from the interface. Shipping a menu that does nothing costs more trust than shipping a shorter menu.

5.5 Gap 5 — Transaction Mathematics Absent

The loan form captures `loan_amount`, `deposit`, `installment` and `due_date` as free inputs. The schema stores them as independent columns with no relationship enforced. There is no amortisation schedule, no derived outstanding balance, and no overdue calculation.

Loan terms must be computed, not typed. A `loan_schedules` table stores one row per installment with due date, amount due, amount paid and status, generated at issuance. Outstanding balance becomes `SUM(scheduled) - SUM(paid)`, which cannot drift the way a hand-maintained field does.

5.6 Gap 6 — AI Layer Has No Data Access

`ai_screen.dart` performs substring matching against local constants. It can recognise the word "sales" but cannot answer "Mauzo ya leo Kariakoo yakoje?" because it has no query path. The AI layer must be rebuilt as a Gemini function-calling client with a read-only tool surface.

6. Target Database Design — Schema Extensions

Four tables are added to `backend/supabase/schema.sql`. Tables 1 and 2 are mandatory and map directly to audit requirements 05 and 13. Tables 3 and 4 close gaps 5 and 12 respectively.

6.1 inventory_movements — Mandatory

Append-only ledger of every device state transition. Immutability enforced by RLS: `INSERT` permitted, `UPDATE` and `DELETE` denied for all roles.

```
CREATE TYPE movement_type AS ENUM (
'intake', 'transfer_out', 'transfer_in', 'sale',
'loan_issue', 'loan_return', 'write_off', 'adjustment'
);

CREATE TABLE IF NOT EXISTS inventory_movements (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
phone_id UUID NOT NULL REFERENCES phones(id),
imei VARCHAR(50) NOT NULL,
movement_type movement_type NOT NULL,
from_branch_id UUID REFERENCES branches(id),
to_branch_id UUID REFERENCES branches(id),
reference_table VARCHAR(100),
reference_id UUID,
employee_id UUID REFERENCES employees(id),
client_operation_id UUID UNIQUE NOT NULL,
notes TEXT,
occurred_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_movements_phone ON inventory_movements(phone_id);
CREATE INDEX idx_movements_imei ON inventory_movements(imei);
CREATE INDEX idx_movements_branch ON inventory_movements(to_branch_id);
CREATE INDEX idx_movements_time ON inventory_movements(occurred_at DESC);
CREATE INDEX idx_movements_client ON inventory_movements(client_operation_id);
```

6.2 sync_operations — Mandatory

Server-side record of every client mutation attempt, enabling idempotent replay and per-device sync diagnostics.

```
CREATE TABLE IF NOT EXISTS sync_operations (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
client_operation_id UUID UNIQUE NOT NULL,
device_id VARCHAR(255) NOT NULL,
employee_id UUID REFERENCES employees(id),
branch_id UUID REFERENCES branches(id),
operation_type VARCHAR(100) NOT NULL,
payload JSONB NOT NULL,
status VARCHAR(50) DEFAULT 'pending',
retry_count INTEGER DEFAULT 0,
last_error TEXT,
client_created_at TIMESTAMP WITH TIME ZONE NOT NULL,
applied_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_sync_status ON sync_operations(status);
```

```
CREATE INDEX idx_sync_device ON sync_operations(device_id);
CREATE INDEX idx_sync_client ON sync_operations(client_operation_id);
```

The unique constraint on `client_operation_id` is the entire idempotency guarantee. A replayed operation hits the constraint, the server recognises the duplicate, and returns the original result rather than double-posting a sale.

6.3 loan_schedules — Recommended

One row per installment, generated at loan issuance. Makes outstanding balance a computed projection rather than a maintained field.

```
CREATE TABLE IF NOT EXISTS loan_schedules (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
loan_id UUID NOT NULL REFERENCES loans(id) ON DELETE CASCADE,
installment_no INTEGER NOT NULL,
due_date DATE NOT NULL,
amount_due DECIMAL(12,2) NOT NULL,
amount_paid DECIMAL(12,2) DEFAULT 0,
status VARCHAR(50) DEFAULT 'pending',
paid_at TIMESTAMP WITH TIME ZONE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
UNIQUE (loan_id, installment_no)
);

CREATE INDEX idx_sched_loan ON loan_schedules(loan_id);
CREATE INDEX idx_sched_due ON loan_schedules(due_date) WHERE status = 'pending';
```

6.4 alert_rules — Recommended

Declarative alert definitions evaluated on a schedule, replacing the static notification array.

```
CREATE TABLE IF NOT EXISTS alert_rules (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
name VARCHAR(255) NOT NULL,
rule_type VARCHAR(100) NOT NULL,
threshold_value DECIMAL(12,2),
branch_id UUID REFERENCES branches(id),
severity VARCHAR(50) DEFAULT 'warning',
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

rule_type	Condition	Severity
`low_stock`	Branch available phone count below `threshold_value`	warning
`overdue_loan`	`loan_schedules.due_date` passed with `amount_paid` < `amount_due`	critical
`stock_discrepancy`	Daily closing physical count differs from expected closing	critical
`missing_report`	No `daily_closing` row for a branch by 21:00 local time	warning
`stale_transfer`	`stock_transfers.status = 'pending'` for more than 48 hours	warning
`inactive_employee`	Employee with no `attendance` check-in for 3 consecutive working days	info

7. Offline-First Sync Architecture

Offline-first is not a feature of the app — it is the shape of the data layer. The design below is the contract both engineers implement against.

7.1 Local Store

A Drift-managed SQLite database mirrors the tables a branch terminal needs to operate without a network: `phones`, `customers`, `branches`, `employees` (read cache), plus write buffers for `sales`, `loans`, `loan_payments`, `attendance` and `stock_transfers`. The mirror is populated on login and refreshed by a realtime subscription when online.

7.2 Write Path — Local First, Always

Every mutation follows the same five steps without exception:

- Generate a `client_operation_id` UUID v4 on the device.
- Write the row into the local SQLite table with `sync_state = 'pending'`.
- Insert a matching row into the local `outbox` table carrying the operation type and full JSON payload.
- Return success to the UI immediately — the operator never waits on the network.
- Wake the sync worker.

7.3 Sync Worker

The worker drains the outbox oldest-first and is safe to interrupt at any point:

- Claim the next batch of pending outbox rows and mark them `in_flight`.
- POST each payload to Supabase with its `client_operation_id` intact.
- On success — mark `synced`, write the server-issued id back to the local row.
- On duplicate-constraint response — treat as success. The operation already landed.
- On network failure — return rows to `pending` with an incremented `retry_count` and exponential backoff.
- On validation failure — mark `failed` and surface it in the sync status widget for manager review.

> **IDEMPOTENCY RULE**
> The server never trusts arrival order. `client_operation_id` is the sole identity of an operation. A sale submitted six times over a flapping connection must produce exactly one row in `sales` — the unique constraint on `sync_operations.client_operation_id` and on `inventory_movements.client_operation_id` enforces this at the database, not in application code.

7.4 Conflict Resolution

Entity	Strategy	Rationale
Sales, loans, attendance	Client wins (append-only)	These are events. Two terminals cannot legitimately record the same event twice.
Phone status & branch	Server wins, re-derived	Device state is a projection of the ledger. Clients never assert device location directly.

Entity	Strategy	Rationale
Customer records	Last-write-wins by server timestamp	Mutable reference data with low contention.
Transfer receipts	Server wins on status	Only the receiving branch may confirm receipt; the originating device cannot overrule it.

7.5 Sync Surface in the UI

`core/widgets/sync_status.dart` already exists in the codebase. It is promoted from decorative to functional: it displays pending operation count, last successful sync time, and offers a manual retry. The operator must always be able to answer the question "is my work safe?" without leaving the screen.

8. Security & Access Control

8.1 Authentication

Supabase Auth replaces the mock login. Sessions are persisted natively by `supabase_flutter`, which handles refresh tokens. The app must handle three session states explicitly: authenticated, refresh-in-progress (never log the user out during a transient refresh), and expired (redirect to login while preserving any unsynced local work — never discard the outbox on logout).

8.2 Row Level Security

RLS is enabled on every table. No table is left open. The policy model derives the caller's branch from their `employees` row rather than trusting any client-supplied branch identifier.

Table group	Policy	Effect
`phones`, `sales`, `loans`, `attendance`, `daily_closing`	Branch-scoped SELECT and INSERT	A branch employee reads and writes only their own branch rows.
`inventory_movements`	INSERT only, SELECT branch-scoped	Immutable. No UPDATE or DELETE policy exists for any role.
`sync_operations`	INSERT and SELECT on own device_id	A device sees only its own queue history.
`employees`, `branches`	SELECT all, UPDATE admin only	Reference reads are open; administration is restricted.
`audit_logs`	SELECT admin only, INSERT via trigger	Written by database triggers, never by clients.
`notifications`	SELECT own branch, UPDATE is_read	Marking read is the only permitted client write.

8.3 Audit Integrity

`audit_logs` must be populated by PostgreSQL triggers on the transactional tables, not by application code. Application-written audit rows can be omitted by a buggy client; trigger-written rows cannot.

8.4 Attendance Anti-Tampering

The check-in screen must send no timestamp. The server applies `NOW()` on insert. The client's device clock is treated as untrusted input and is never persisted as an authoritative time. Offline check-ins carry the device time in a distinct column for later reconciliation, never overwriting `check_in`.

9. Implementation Roadmap

Delivery is sequenced into four phases. Phase 1 is a hard prerequisite for Phases 2 through 4 — nothing downstream can be validated without a live data layer.

Phase	Focus	Exit Criteria
Phase 1	Database foundation & Supabase integration	Live login, live branch stock list, ledger and sync tables deployed, repository layer switched by environment flag.
Phase 2	Transaction flows — sales, loans, transfers	A sale and a loan complete end to end against the database; box receiving reconciles a manifest with IMEI uniqueness enforced.
Phase 3	Stocktake, daily closing & reporting	Physical stocktake produces a real discrepancy list; daily close reconciles; reports export to CSV and PDF from live data.
Phase 4	AI intelligence layer	Gemini answers stock, sales and IMEI-location questions from live data with read-only tool access.

9.1 Phase 1 — Database Foundation & Supabase Integration

Objective: convert the application from a demonstration into a system with a real spine.

- Deploy `schema.sql` to a Supabase project; apply the Section 6 extensions in the same migration.
- Seed `branches` and `employees` with the real operating branches.
- Replace both placeholder credential constants with environment-injected values; add `.env` loading and keep the file out of version control.
- Introduce `lib/core/data/repository.dart` defining `LimsRepository` as an abstract interface.
- Implement `SupabaseRepository` against it, mapping each method to the queries already written in `backend/lib/supabase_client.dart`.
- Implement `MockRepository` with seeded data for offline demonstration and widget tests.
- Bind `apiServiceProvider` to select the implementation by build flag.
- Build the Drift local store and the outbox table.
- Enable RLS on all tables with the policies in Section 8.2.

> **PHASE 1 GATE**
> A branch manager signs in with real credentials, sees their real branch stock count from PostgreSQL, and the sync status widget reports zero pending operations. Nothing fabricated remains in the read path.

9.2 Phase 2 — Transaction Flows

- Implement the IMEI scan input accepting three sources: manual entry, camera barcode scan via `mobile_scanner`, and USB wedge-mode scanner as keyboard input.
- Make intake transactional: a box is received as a batch, each IMEI validated as unique against `phones` before any row is committed.
- Generate the expected-versus-received reconciliation that `receive_stock_screen.dart` already renders, from real submitted data.
- Implement loan amortisation: given principal, deposit, term and rate, generate `loan_schedules` rows at issuance.

- Route every repayment through `loan_payments` and update the matching schedule row, recomputing outstanding balance as a projection.
- Write one `inventory_movements` row for every device state change, with the originating `client_operation_id`.
- Make transfers two-phase: dispatch records `transfer_out`; receipt records `transfer_in` and moves the device. The dispatch alone never relocates stock.

> **PHASE 2 GATE**
> A complete sale is performed on a device with Wi-Fi disabled, the app reports the sale as locally saved, the network is restored, and the row appears in Supabase exactly once with a matching ledger entry.

9.3 Phase 3 — Stocktake, Daily Closing & Reporting

- Implement the physical stocktake scanner producing three output sets: missing from shelf but present in database, present on shelf but absent from database, and present in the wrong branch.
- Drive daily closing arithmetic from the ledger rather than from form inputs: opening from the previous close, received and transferred from movements, sold and loaned from transactions.
- Wire the four dead report actions to real generators.
- Add `pdf` and `csv` export. CSV for operational handoff, PDF for management review.
- Replace chart literals in `reports_screen.dart` and `branch_performance_screen.dart` with live aggregate queries.
- Implement the alert evaluator over `alert_rules` on a Supabase scheduled function.

> **PHASE 3 GATE**
> A stocktake deliberately removes two IMEIs from the shelf; the closing report names both devices and flags the discrepancy as critical, and the manager exports the report as a PDF.

9.4 Phase 4 — AI Intelligence Layer

The assistant is rebuilt as a Gemini function-calling client. The critical design constraint is that the tool surface is **read-only by construction** — there is no code path from the model to a write.

Tool	Answers	Backing Query
`get_branch_sales`	"Mauzo ya leo Kariakoo yakoje?"	`sales` aggregated by branch and date range
`get_low_stock`	"Ni simu zipi zimebaki chini ya 5?"	`phones` grouped by model where available count < threshold
`locate_imei`	"Simu yenye IMEI hii iko wapi?"	`phones` joined to `branches`, plus movement history from the ledger
`get_overdue_loans`	"Mikopo ipi imechelewa?"	`loan_schedules` where due_date < today and unpaid
`get_attendance_summary`	"Nani yupo leo?"	`attendance` for the current date, scoped to caller branch
`get_stock_valuation`	"Thamani ya stock ni kiasi gani?"	`phones` at cost, grouped by branch

- Every tool executes under the caller's RLS context — the model can never see another branch's data.

- Tool results are capped in row count and returned as compact summaries to keep the context small.
- The system prompt forbids asserting any figure not returned by a tool.
- A refusal path is mandatory: when no tool can answer, the assistant says so rather than estimating.

10. Delivery Ownership Matrix

Both developers own the outcome. Workstreams are assigned a lead to prevent parallel work on the same file, with the other engineer as reviewer.

Workstream	Lead	Reviewer	Primary Artefacts
Schema migrations & RLS policies	Godfrey Shora	David Mwijage	schema.sql, migration files, policy scripts
Repository layer & Supabase wiring	David Mwijage	Godfrey Shora	repository.dart, supabase_repository.dart, mock_repository.dart
Local Drift store & outbox	Godfrey Shora	David Mwijage	local database, outbox table, migrations
Sync worker & idempotency	Godfrey Shora	David Mwijage	sync_worker.dart, sync_status wiring
Scanning input (camera + USB wedge)	David Mwijage	Godfrey Shora	scan_screen.dart, mobile_scanner integration
Loan amortisation engine	David Mwijage	Godfrey Shora	loan_schedule_generator.dart
Ledger emission on all mutations	Godfrey Shora	David Mwijage	movement writer, RLS insert policies
Stocktake & daily closing logic	David Mwijage	Godfrey Shora	stocktake diff, daily_closing computation
Reporting & PDF/CSV export	Godfrey Shora	David Mwijage	report generators, export service
Alert rules engine	Godfrey Shora	David Mwijage	alert_rules seed, evaluator function
Gemini function-calling assistant	David Mwijage	Godfrey Shora	ai tools, tool executor, system prompt
Integration test suite	Godfrey Shora	David Mwijage	integration_test/ — sale, transfer, stocktake

11. Acceptance Criteria & Test Coverage

No workstream is complete on demonstration alone. Each is accepted against a reproducible test.

ID	Scenario	Pass Condition
AC-01	Live authentication	Real credentials produce a session; the branch is resolved server-side from the employee record.
AC-02	Branch stock read	Stock count shown equals `SELECT COUNT(*) FROM phones WHERE branch_id = <caller branch>`.
AC-03	Offline sale	Wi-Fi disabled, sale completes locally, sync status shows one pending operation.
AC-04	Replay safety	Same sale submitted six times yields exactly one `sales` row.
AC-05	IMEI uniqueness	Submitting a duplicate IMEI is rejected before any row is written.
AC-06	Batch receiving	Manifest of 20 IMEIs with 18 delivered produces exactly two discrepancies, both itemised.
AC-07	Loan schedule	A 6-month loan generates exactly 6 schedule rows whose amounts sum to principal less deposit.
AC-08	Balance projection	Outstanding balance always equals scheduled total minus paid total.
AC-09	Ledger completeness	Every mutation produces exactly one `inventory_movements` row carrying its `client_operation_id`.
AC-10	Ledger immutability	An UPDATE or DELETE against `inventory_movements` fails under RLS for every role tested.
AC-11	Two-phase transfer	Dispatch alone does not move a device; receipt does.
AC-12	Stocktake diff	Two deliberately hidden IMEIs appear in the discrepancy list.
AC-13	Daily closing	Expected closing equals opening plus received plus transferred-in minus sold minus loaned minus transferred-out.
AC-14	Report export	CSV and PDF both generated from live query results, not literals.
AC-15	Alert fidelity	Dropping stock below threshold raises exactly one notification.
AC-16	AI grounding	Asked a question with no matching tool, the assistant declines rather than inventing a figure.
AC-17	AI read-only	Automated audit confirms no AI tool issues an INSERT, UPDATE or DELETE.
AC-18	Attendance integrity	A device clock set two hours forward does not alter the stored `check_in`.
AC-19	Cross-branch isolation	A branch employee cannot read another branch's sales under RLS.
AC-20	Session resilience	A transient token refresh does not log the operator out or discard the outbox.

12. Risk Register & Mitigations

Risk	Likelihood	Impact	Mitigation
Connectivity at branch level is worse than assumed	High	High	Offline-first is Phase 1, not a later enhancement. The app must be fully usable with the network permanently down.
Duplicate sales from replayed operations	Medium	Critical	Database-level unique constraint on `client_operation_id`. Never rely on application-side deduplication.
Device location drift between dispatch and receipt	High	Medium	Two-phase transfer with the ledger as the single source of truth. No direct writes to `phones.branch_id`.
Scope creep into the AI layer before data is live	Medium	Medium	Phase 1 gate is enforced. The AI layer has no data to ground on until the repository is live.
RLS misconfiguration exposing cross-branch data	Medium	Critical	Policy tests are part of acceptance (AC-19). Branch identity derives from the employee record, never from client input.
Bundle bloat from the new dependency set	Low	Low	Drift, mobile_scanner, pdf and an HTTP client are all lazy-initialised; cold start target remains under two seconds.

13. Muhtasari wa Kiswahili

Sehemu hii ni muhtasari mfupi kwa Kiswahili kwa ajili ya mazungumzo ya ndani na timu ya uendeshaji wa matawi.

13.1 Hali ya Sasa ya App

- Screens zote za tawi na za mkuu zipo na zinafanya kazi vizuri — UI ni imara.
- Lakini data yote ni ya kufikirika. `api_service.dart` inarudisha majibu ya kubuni tu, hakuna muunganisho wa Supabase.
- Hakuna hifadhidata ya ndani (local DB), hivyo app haiwezi kufanya kazi bila internet.
- Vitufe 13 kwenye Admin na Reports havina kazi bado — `onTap: () {}`.
- Meza mbili muhimu hazipo: `inventory_movements` (kumbukumbu ya kila mabadiliko ya simu) na `sync_operations` (foleni ya kusawazisha data).

13.2 Mpango wa Utekelezaji

Awamu	Kazi Kuu	Matokeo
Awamu 1	Kuunganisha database na Supabase, kuunda repository layer, kuongeza meza za ledger na sync	Login halisi, stock halisi kutoka database, msingi wa offline-first
Awamu 2	Miamala — mauzo, mikopo, uhamisho wa stock, kupokea box	Mauzo na mkopo yanakamilika kwa data halisi; IMEI uniqueness inalindwa
Awamu 3	Stocktake, kufunga siku na ripoti	Orodha halisi ya tofauti; ripoti zinaweza kutolewa kama CSV na PDF
Awamu 4	AI Assistant kwa Gemini	Meneja anaweza kuuliza kwa Kiswahili na kupata jibu kutoka data halisi

13.3 Kanuni ya Msingi

Kila mabadiliko ya simu yanatakiwa kuandikwa kwenye `inventory_movements`. Kila operesheni inayotoka kwenye simu inatakiwa iwe na `client_operation_id` ya kipekee ili tusipate mauzo yaliyorudiwa mara mbili. Na server timestamp pekee ndiyo inahesabika kwa mahudhurio — saa ya simu haiaminiki.

Appendix A — Source File Inventory

Files inspected during this audit, with counts verified by direct enumeration.

Path	Type	Note
`frontend/lib/main.dart`	Dart	Entry point, Riverpod scope
`frontend/lib/core/routing/app_router.dart`	Dart	12 routes, 8 outside the shell
`frontend/lib/core/services/api_service.dart`	Dart	75 lines — mock provider, all live calls commented
`frontend/lib/core/theme/app_theme.dart`	Dart	Design tokens
`frontend/lib/core/theme/app_localizations.dart`	Dart	Swahili and English strings
`frontend/lib/core/constants/app_constants.dart`	Dart	Static app constants
`frontend/lib/core/widgets/`	Dart	16 widgets including sync_status, discrepancy_card, metric_card
`frontend/lib/features/admin/presentation/admin_dashboard_screen.dart`	Dart	9 dead `onTap: () {}` targets
`frontend/lib/features/reports/presentation/reports_screen.dart`	Dart	4 dead `onTap: () {}` targets
`frontend/lib/features/ai/presentation/ai_screen.dart`	Dart	Keyword matching only; no API client
`frontend/lib/features/inventory/presentation/receive_stock_screen.dart`	Dart	Static BX-1024 sample
`frontend/lib/features/sales/presentation/sell_loan_screen.dart`	Dart	Forms present, no amortisation
`backend/lib/supabase_client.dart`	Dart	96 lines — correct queries, placeholder credentials
`backend/supabase/schema.sql`	SQL	177 lines — 12 tables, 8 indexes
`frontend/pubspec.yaml`	YAML	4 runtime dependencies
`frontend/test/widget_test.dart`	Dart	Default scaffold only

Metric	Value
Dart source files (front end)	37
Total front-end application code	~6,430 lines
Feature screens	15
Reusable widgets	16
Dead navigation targets	13
TODO markers in front end	8
Database tables (current)	12
Database tables (target)	16
Runtime dependencies (current)	4
Runtime dependencies (target)	~9

Appendix B — Dependency Additions

Additions required in `frontend/pubspec.yaml`. Current declarations beyond the Flutter SDK are `flutter_riverpod`, `go_router`, `intl` and `supabase_flutter`.

```
dependencies:
# --- persistence: offline-first local store
drift: ^2.16.0
sqlite3_flutter_libs: ^0.5.20
path_provider: ^2.1.2

# --- scanning: camera barcode + USB wedge input
mobile_scanner: ^5.1.0

# --- reporting: PDF and CSV export
pdf: ^3.10.8
printing: ^5.12.0
csv: ^6.0.0

# --- AI: Gemini function calling
google_generative_ai: ^0.4.3
http: ^1.2.1

# --- connectivity awareness for the sync worker
connectivity_plus: ^6.0.3

dev_dependencies:
drift_dev: ^2.16.0
build_runner: ^2.4.9
integration_test:
sdk: flutter
```

Appendix C — Glossary

Term	Definition
IMEI	International Mobile Equipment Identity — the 15-digit unique identifier of a physical handset. The system's primary unit of inventory.
inventory_movements	Append-only ledger recording every state transition of every device. Immutable by design.
client_operation_id	UUID generated on the device before any network call. The sole identity of an operation, guaranteeing replay safety.
Outbox	Local durable queue of unsynchronised mutations, drained by the sync worker when connectivity returns.
Sync worker	Background process that replays the outbox to Supabase and reconciles local state with server state.
Idempotency	The property that repeating an operation produces the same result as performing it once. Enforced by unique constraints.
Amortisation schedule	Table of installments with due dates and amounts, generated at loan issuance.
Row Level Security (RLS)	PostgreSQL policy system restricting which rows a database role may read or write. The primary isolation mechanism.
Two-phase transfer	Transfer protocol where dispatch and receipt are separate recorded events. Stock relocates only on receipt.
Wedge mode	USB scanner behaviour where the device presents as a keyboard, typing the scanned barcode into the focused field.
Function calling	The mechanism by which an LLM requests execution of a declared function with structured arguments. Used to ground the AI assistant in live data.
Projection	A value derived from recorded events rather than stored directly. Device location and loan balance are both projections in this design.

Sign-Off & Approval

This document is the controlling technical specification for the Loan & Inventory Management System. Deviations require recorded amendment.

Role	Name	Responsibility	Status
Lead Developer	Godfrey Shora	Core implementation, Database & RLS, offline sync, reporting, test suite	Approved
Architecture & Systems	David Erick Mwijage	System architecture, repository layer, transaction logic, AI integration	Approved
Project Ownership	Godfrey Shora · David Erick Mwijage	Technical governance and release authority	Approved

NEXT ACTION

Begin Phase 1: deploy `schema.sql` with the Section 6 extensions to a Supabase project, and create `lib/core/data/repository.dart`. Every subsequent phase depends on that gate being passed.